Laboratorium 6 - Three.js: personalizacja i rozbudowa gry 3D

1. Wprowadzenie

Ostatnie laboratorium będzie miało charakter **samodzielnego warsztatu projektowego**. Celem jest rozbudowanie przygotowanej gry o dodatkowe efekty wizualne, animacje oraz elementy interfejsu, które poprawią jej wygląd i odbiór rozgrywki.

2. Płynny obrót i animacja skoku postaci

2.1. Płynny obrót postaci z wykorzystaniem interpolacji

2.1.1. Wykorzystamy metodę THREE.MathUtils.lerp(), która pozwala stopniowo zmieniać wartość pomiędzy pozycją początkową a docelową:

```
THREE.MathUtils.lerp(start, end, alpha);
```

Parametr alpha określa stopień interpolacji: 0 oznacza wartość początkową, a 1 docelową.

2.1.2. W pliku *main.js*, bezpośrednio po utworzeniu modelu playerModel, dodaj zmienną przechowującą docelowy kąt obrotu:

```
let targetRotation = playerModel.rotation.y;
```

2.1.3. W istniejącej obsłudze keydown odszukaj fragment odpowiedzialny za obrót postaci po wykonaniu ruchu:

```
playerModel.rotation.y = angles[event.key];
```

Zastąp go instrukcją:

```
targetRotation = angles[event.key];
```

2.1.4. W istniejącej funkcji **frame(time)**, wewnątrz bloku aktualizującego aktywną rozgrywkę, dodaj:

```
const angle = playerModel.rotation.y;
const difference = Math.atan2(
   Math.sin(targetRotation - angle),
   Math.cos(targetRotation - angle)
);

playerModel.rotation.y = THREE.MathUtils.lerp(
   angle,
   angle + difference,
   1 - Math.exp(-15 * dt)
);
```

Funkcje atan2(), sin() i cos() wyznaczają najkrótszy kierunek obrotu, natomiast lerp() stopniowo zbliża model do docelowego kąta. Współczynnik 15 określa szybkość obrotu, a uwzględnienie dt zapewnia płynność niezależnie od liczby klatek.

2.2. Prosta animacja skoku postaci

2.2.1. Wprowadzimy krótką animację skoku, podczas której postać będzie płynnie przemieszczać się pomiędzy kolejnymi polami planszy. Zamiast natychmiastowej zmiany położenia modelu wykorzystamy interpolację liniową (lerp) do przesuwania go w osiach X i Z oraz funkcję sinus do uzyskania efektu uniesienia i opadania w osi Y. Pod zmienną targetRotation dodaj:

```
const JUMP_DURATION = 0.25;
let jumpTime = 0;
let jumpStartX = 0;
let jumpStartZ = 0;
```

Stała JUMP_DURATION określa czas trwania skoku w sekundach. Zmienna jumpTime przechowuje pozostały czas animacji, natomiast jumpStartX i jumpStartZ zapamiętują początkowe przesunięcie modelu względem docelowej pozycji gracza.

2.2.2. W istniejącej obsłudze **keydown**, wewnątrz warunku sprawdzającego, czy pozycja gracza rzeczywiście się zmieniła, bezpośrednio po instrukcji targetRotation = angles[event.key]; dodaj:

```
jumpStartX = playerModel.position.x + oldX - scene.player.position[0];
jumpStartZ = playerModel.position.z + oldZ - scene.player.position[2];
jumpTime = JUMP_DURATION;
```

Zapamiętane przesunięcie pozwala rozpocząć skok z poprzedniego położenia modelu, również gdy kolejny ruch nastąpi przed zakończeniem animacji.

2.2.3. W funkcji frame(time), wewnątrz bloku aktualizującego aktywną rozgrywkę, pod fragmentem interpolującym obrót postaci dodaj:

```
if (jumpTime > 0) {
   jumpTime = Math.max(0, jumpTime - dt);
   const progress = 1 - jumpTime / JUMP_DURATION;
   playerModel.position.x = THREE.MathUtils.lerp(jumpStartX, 0, progress);
   playerModel.position.z = THREE.MathUtils.lerp(jumpStartZ, 0, progress);
   playerModel.position.y = Math.sin(progress * Math.PI) * 0.12;
}
```

Zmienna progress określa postęp animacji od 0 do 1. Interpolacja przesuwa model między polami, a sinus nadaje ruchowi kształt łuku o maksymalnej wysokości 0.12. Zmieniana jest wyłącznie pozycja wizualna modelu, bez modyfikacji logiki kolizji.2.2.4. W istniejącej obsłudze restartu klawiszem R, po wywołaniu restartGame(), dodaj:

```
jumpTime = 0;
playerModel.position.y = 0;
```

Dzięki temu po restarcie animacja zostaje zakończona, a model wraca do swojej pozycji lokalnej. Analogiczne wyzerowanie animacji należy wykonać po utracie życia, gdy gracz zostaje przeniesiony na początkowy pas.

3. Poszerzenie terenu planszy

Rozszerzymy istniejące pasy drogi i trawy w osi X, zachowując dotychczasowe granice poruszania się gracza. Dzięki temu otoczenie będzie wykraczać poza obszar rozgrywki, a plansza sprawi wrażenie bardziej rozbudowanej.

3.1. W pliku objects.js zwiększ szerokość geometrii pasów:

```
const rowGeometry = new THREE.BoxGeometry(5.0, 0.04, 0.35);
```

3.2. W funkcji addRowVisual() rozszerz zakres tworzenia przerywanych oznaczeń jezdni:

```
for (let x = -2.25; x <= 2.25; x += 0.30) {
```

Zmiany obejmą zarówno początkowe, jak i dynamicznie generowane pasy drogi i trawy.

3.3. W pliku gameLogic.js pozostaw dotychczasowe ograniczenie ruchu gracza (limitX = 0.75). Poszerzenie terenu nie powinno wpływać na dostępny obszar poruszania się postaci.

3.4. Dostosuj również mechanizm ruchu samochodów do nowych wymiarów planszy, wykorzystując rozwiązania z poprzedniego laboratorium. Pojazdy powinny pojawiać się i znikać poza bocznymi krawędziami drogi. Możesz również zwiększyć ich maksymalną liczbę do 3 na pas, zachowując wspólną prędkość i kierunek jazdy oraz odpowiednie odstępy między samochodami.

4. Zadania - zasady realizacji

Zapoznaj się z poniższymi propozycjami i **wybierz trzy pomysły**, które zaimplementujesz w swojej grze. Każdy pomysł zawiera opis, przykładowe fragmenty kodu oraz oczekiwany efekt.

Ważne: Przedstawione fragmenty kodu mają charakter pomocniczy i nie stanowią kompletnych rozwiązań. Należy przeanalizować ich działanie, umieścić w odpowiednich miejscach projektu oraz dostosować do istniejących funkcji i zmiennych. Nie należy tworzyć ponownie elementów, które już istnieją w aplikacji.

Propozycje rozbudowy gry

Pomysł 1. Cykl dnia i nocy

Wprowadź możliwość zmiany pory dnia w grze. Przygotuj przynajmniej dwa warianty, różniące się kolorystyką światła i intensywnością oświetlenia.

Zmiana pory dnia powinna następować na podstawie jednego z wybranych mechanizmów:

- Upływu czasu np. zmiana pory dnia co 20 sekund.
- Liczby wykonanych przeskoków np. zmiana co 50 pokonanych pól.

Przejścia pomiędzy porami dnia powinny być płynne, z wykorzystaniem interpolacji (lerp). Utrata życia nie może resetować cyklu aktualna pora dnia i postęp do kolejnej zmiany powinny zostać zachowane.

W pliku main.js wykorzystaj istniejące oświetlenie sceny. Przykładowe wartości dla obu wariantów:

```
const dayColor = new THREE.Color(0x87bfe5);
const nightColor = new THREE.Color(0x0c1629);
const dayLight = 2.0;
const nightLight = 0.3;
```

W przypadku koloru tła zastosuj interpolację kolorów:

```
scene3D.background.lerpColors(dayColor, nightColor, alpha);
```

Parametr alpha powinien zmieniać się stopniowo od 0 do 1, określając postęp przejścia między dniem a nocą.

Uwaga: Materiał MeshBasicMaterial nie reaguje na oświetlenie. Aby zmiana pory dnia wpływała również na teren, rozważ zastosowanie MeshStandardMaterial dla pasów drogi i trawy.

Oczekiwany efekt: Możliwość zmiany wyglądu świata pomiędzy co najmniej dwoma porami dnia.

Dokumentacja: AmbientLight, DirectionalLight, MeshStandardMaterial

Pomysł 2. Cienie i ulepszone oświetlenie

Wprowadź cienie rzucane przez modele samochodów, drzew i postaci. Dostosuj materiały terenu, aby reagowały na oświetlenie sceny.

W pliku main.js, po utworzeniu odpowiednich obiektów, włącz obsługę cieni dla renderera i istniejącego światła kierunkowego:

```
renderer3D.shadowMap.enabled = true;
light.castShadow = true;
light.shadow.mapSize.set(1024, 1024);
```

W pliku objects.js zmień materiały pasów z MeshBasicMaterial na MeshStandardMaterial, a wewnątrz funkcji addRowVisual() ustaw:

```
mesh.receiveShadow = true;
Aby modele GLB rzucały cienie, w funkcjach odpowiedzialnych za ich tworzenie
wykorzystaj:
root.traverse(part => {
   if (part.isMesh) {
      part.castShadow = true;
   }
});
```

Samodzielnie dostosuj położenie źródła światła i obszar kamery cieni (light.shadow.camera). Pamiętaj, że plansza jest generowana dynamicznie, dlatego cienie powinny być widoczne również podczas przemieszczania się na kolejne pasy.

Oczekiwany efekt: Samochody, drzewa i postać rzucają widoczne cienie, które przemieszczają się wraz z obiektami.

Uwaga: Zaleca się, aby nie wybierać jednocześnie Pomysłu 1 (Cykl dnia i nocy) oraz Pomysłu 2, ponieważ oba koncentrują się na oświetleniu sceny. Zachęcamy do wyboru zróżnicowanych rozwiązań, obejmujących różne aspekty gry, np. wygląd otoczenia, animacje lub interakcje.

Dokumentacja: WebGLRenderer, DirectionalLightShadow.

Pomysł 3. Materiały i tekstury terenu

Zastąp jednolite kolory pasów drogi i trawy teksturami, aby nadać otoczeniu bardziej realistyczny i szczegółowy wygląd. Dobierz odpowiednie obrazy przedstawiające np. trawę i asfalt.

W katalogu public/textures umieść plik grass.jpg. Następnie w objects.js, poza funkcją addRowVisual(), przygotuj teksturę:

```
const grassTexture = new THREE.TextureLoader().load('/textures/grass.jpg');
grassTexture.colorSpace = THREE.SRGBColorSpace;
grassTexture.wrapS = THREE.RepeatWrapping;
grassTexture.wrapT = THREE.RepeatWrapping;
grassTexture.repeat.set(4, 1);
```

Właściwości wrapS i wrapT umożliwiają powtarzanie obrazu na powierzchni obiektu, natomiast repeat.set(4, 1) określa liczbę powtórzeń w obu kierunkach. Następnie, po deklaracji rowMaterials, przypisz teksturę do materiału trawy:

```
rowMaterials.grass = new THREE.MeshStandardMaterial({
   map: grassTexture
});
```

Samodzielnie przygotuj analogiczne rozwiązanie dla asfaltu. Dobierz sposób powtarzania tekstur tak, aby nie były nadmiernie rozciągnięte i dobrze wyglądały na poszerzonych pasach.

Ważne: Tekstury należy wczytać tylko raz, poza funkcją addRowVisual(), a następnie wykorzystywać je dla wszystkich kolejnych pasów, również generowanych dynamicznie.

Oczekiwany efekt: Pasy trawy i drogi wykorzystują odpowiednio dobrane tekstury, które zachowują prawidłowy wygląd podczas rozbudowy planszy.

Dokumentacja: TextureLoader, MeshStandardMaterial.

Pomysł 4. Dodatkowe dekoracje otoczenia

Rozbuduj świat gry o dodatkowe dekoracje, wykorzystując gotowe modele 3D w formacie .glb, np. znaki drogowe, latarnie, kamienie, kwiaty lub budynki. Elementy powinny być odpowiednio rozmieszczane na kolejnych pasach, urozmaicając otoczenie.

Wybierz model 3D i umieść go w katalogu public/models, np. sign.glb. W pliku objects.js wykorzystaj znane z poprzednich laboratoriów funkcje loadTemplate() i makeInstance():

```
import { loadTemplate, makeInstance } from './modelLoader.js';
const signTemplatePromise = loadTemplate('/models/sign.glb');
```

Model jest wczytywany jednokrotnie i może być wielokrotnie wykorzystywany. Następnie w funkcji addRowVisual(), przed dodaniem grupy root do sceny, możesz zastosować:

```
if (row.type === 'road' && row.laneIndex % 3 === 0) {
  signTemplatePromise.then(template => {
    if (!root.parent) return;

    const sign = makeInstance(template, 0.5);
    sign.position.set(1.2, GROUND_Y, row.z);
    root.add(sign);
  });
}
```

Warunek row.laneIndex % 3 === 0 powoduje umieszczanie dekoracji na co trzecim pasie drogi. Funkcja makeInstance() tworzy kolejną instancję modelu, natomiast root.add() przypisuje dekorację do danego pasa, dzięki czemu zostanie ona usunięta razem z nim.

Samodzielnie dobierz dodatkowe modele, ich skalę, położenie oraz częstotliwość występowania. Zadbaj o to, aby dekoracje nie utrudniały poruszania się gracza i były generowane również na kolejnych fragmentach planszy.

Oczekiwany efekt: Urozmaicone otoczenie wykorzystujące importowane modele 3D, których rozmieszczenie jest powiązane z proceduralnym generowaniem terenu.

Dokumentacja: Group, Mesh,GLTFLoader .

Pomysł 5. Efekt zderzenia z samochodem

Dodaj krótki efekt wizualny sygnalizujący kolizję z samochodem, np. miganie postaci, chwilową zmianę koloru modelu lub drgnięcie kamery.

W pliku main.js, poza funkcją frame(time), zadeklaruj zmienną określającą czas trwania efektu:

```
let flashRemaining = 0;
```

W istniejącej funkcji frame(time) wykorzystaj zmienną livesBefore, przechowującą liczbę żyć przed wywołaniem updateGameState(). Bezpośrednio po tym wywołaniu dodaj:

```
if (gameState.lives < livesBefore) {
  flashRemaining = 0.4;
}
```

Następnie w tej samej funkcji, przed wywołaniem render(), dodaj:

```
flashRemaining = Math.max(0, flashRemaining - dt);
playerModel.visible = flashRemaining === 0 || Math.floor(flashRemaining * 20) % 2 === 0;
```

Zmienna flashRemaining odlicza czas efektu, natomiast zmiana właściwości visible powoduje naprzemienne wyświetlanie i ukrywanie modelu, tworząc efekt migania.

Ważne: Przy restarcie zadbaj o wyzerowanie efektu i przywrócenie widoczności postaci.

Przedstawiony kod stanowi przykład. Możesz przygotować własny efekt zderzenia lub połączyć kilka efektów, dostosowując ich czas trwania i intensywność.

Oczekiwany efekt: Kolizja z samochodem jest wyraźnie sygnalizowana krótką animacją, która nie zakłóca dalszej rozgrywki ani restartu.

Pomysł 6. Efekty cząsteczkowe (Particle Effects)

Dodaj do gry efekt cząsteczkowy, np. padający śnieg, unoszący się kurz, deszcz lub opadające liście. Efekt powinien być animowany i dostosowany do otoczenia oraz ruchu kamery.

W Three.js do tworzenia cząsteczek możesz wykorzystać klasę THREE.Points, która pozwala wyświetlać wiele punktów za pomocą jednej geometrii i materiału.

Przykładowo w pliku main.js przygotuj geometrię i materiał cząsteczek:

```
const geometry = new THREE.BufferGeometry();
geometry.setAttribute(
   'position',
   new THREE.Float32BufferAttribute(positions, 3)
);

const particles = new THREE.Points(
   geometry,
   new THREE.PointsMaterial({
      color: 0xffffff,
      size: 3,
      sizeAttenuation: false
   })
);
scene3D.add(particles);
```

Tablica positions powinna zawierać współrzędne początkowe cząsteczek w postaci kolejnych wartości X, Y i Z. Przygotuj ją samodzielnie, wykorzystując np. Math.random() do losowego rozmieszczenia punktów. Aby uzyskać efekt ruchu, w istniejącej funkcji frame(time) aktualizuj pozycje cząsteczek. Przykładowy fragment odpowiedzialny za opadanie:

```
const attribute = geometry.attributes.position;
for (let i = 0; i < attribute.count; i++) {
   const y = attribute.getY(i) - dt * 0.4;
   attribute.setY(i, y < 0 ? 2 : y);
}
attribute.needsUpdate = true;
```

Samodzielnie dostosuj kierunek i prędkość ruchu, obszar występowania cząsteczek oraz sposób ich ponownego pojawiania się. Uwzględnij przemieszczanie się kamery i zatrzymanie animacji podczas pauzy.

Ważne: Cząsteczki powinny być wykorzystywane wielokrotnie, zamiast tworzenia kolejnych obiektów w każdej klatce animacji.

Oczekiwany efekt: Płynny efekt cząsteczkowy bez niekontrolowanego wzrostu liczby obiektów.

Dokumentacja: Points, BufferGeometry.

Pomysł 7. Rozbudowane menu i interfejs rozgrywki

Przygotuj spójny interfejs gry obejmujący menu główne, ekran pauzy oraz ekran zakończenia rozgrywki. Poszczególne ekrany powinny zawierać odpowiednie przyciski, informacje o grze oraz estetyczną oprawę wizualną.

Menu główne powinno umożliwiać rozpoczęcie gry i zapoznanie się ze sterowaniem. Ekran pauzy powinien pozwalać na wznowienie lub ponowne rozpoczęcie rozgrywki, natomiast po utracie wszystkich żyć należy wyświetlić podsumowanie z uzyskanym wynikiem i rekordem.

W pliku index.html przygotuj przykładową strukturę menu:

```
<div id="startMenu" class="game-menu">
  <h1>Road Hopper 3D</h1>
  <p>Przedostań się jak najdalej, unikając przeszkód!</p>

  <button id="startButton">Rozpocznij grę</button>
  <!-- Sterowanie, ustawienia i inne elementy -->
</div>
```

W main.js przygotuj zmienną określającą aktualnie wyświetlany ekran. Możesz wykorzystać następujące stany:

```
let currentScreen = 'menu'; // menu | playing | paused | gameover
```

Przykładowy mechanizm przełączania ekranów:

```
function showScreen(screen) {
  currentScreen = screen;

  document.querySelectorAll('.game-menu').forEach(menu => {
    menu.hidden = menu.id !== `${screen}Menu`;
  });
}
```

Samodzielnie przygotuj obsługę przycisków i powiąż stany interfejsu z istniejącymi mechanizmami gry (isPaused, gameState.status, restartGame()). Pamiętaj, aby zatrzymać aktualizację rozgrywki i sterowanie postacią, gdy wyświetlane jest menu lub ekran pauzy. Renderowanie sceny może nadal działać w tle.

Ważne: Nie twórz drugiej pętli animacji ani dodatkowych mechanizmów restartu. Rozbuduj istniejącą obsługę gry i zadbaj o poprawne przechodzenie pomiędzy ekranami. Przy korzystaniu z atrybutu hidden upewnij się, że style CSS rzeczywiście ukrywają nieaktywne menu.

Oczekiwany efekt: Gra posiada estetyczne, funkcjonalne menu z ustawieniami, ekran pauzy oraz podsumowanie rozgrywki, a przejścia między nimi nie zakłócają działania aplikacji.

Pomysł 8. Efekty dźwiękowe

Dodaj dźwięki związane z wydarzeniami w grze, np. ruchem gracza, zderzeniem z samochodem czy zdobyciem punktów.

Umieść pliki dźwiękowe w katalogu public/sounds, a w pliku main.js przygotuj:

```
const sounds = {
  jump: new Audio('/sounds/jump.mp3'),
  hit: new Audio('/sounds/hit.mp3'),
  score: new Audio('/sounds/score.mp3')
};

function playSound(sound) {
  sound.currentTime = 0;
  sound.play().catch(error => console.warn(error));
}
```

Przy odpowiednim zdarzeniu wywołuj np.:

```
playSound(sounds.jump);
```

Samodzielnie powiąż odtwarzanie dźwięków z istniejącymi zdarzeniami gry. Dodaj również możliwość wyciszania i ponownego włączania dźwięków klawiszem M, wykorzystując istniejącą obsługę keydown oraz właściwość muted obiektów audio.

Pamiętaj, aby nie uruchamiać dźwięków w każdej klatce animacji. Przeglądarka może wymagać wcześniejszej interakcji użytkownika przed ich odtworzeniem.

Oczekiwany efekt: Co najmniej trzy różne efekty dźwiękowe oraz możliwość ich wyciszania i włączania podczas rozgrywki.

Pomysł 9. Animowane komunikaty podczas rozgrywki

Dodaj krótkie, animowane komunikaty informujące o ważnych wydarzeniach, np. zdobyciu punktów, pobiciu rekordu czy utracie życia.

W pliku index.html przygotuj element do wyświetlania powiadomień:

```
<div id="notification"></div>
```

W main.js możesz wykorzystać właściwość textContent do zmiany treści oraz classList do uruchamiania animacji CSS:

```
notification.textContent = message;
notification.classList.add('show');
```

Samodzielnie przygotuj funkcję wyświetlającą komunikaty, określ momenty jej wywoływania oraz zaprojektuj animacje pojawiania się i znikania za pomocą @keyframes. Zadbaj o

odpowiednie rozmieszczenie powiadomień i obsługę sytuacji, gdy kilka wydarzeń nastąpi w krótkim odstępie czasu.

Uwaga: Wykorzystaj osobny element interfejsu, ponieważ istniejący statusLabel jest regularnie aktualizowany przez updateHud().

Oczekiwany efekt: Ważne wydarzenia w grze są sygnalizowane estetycznymi, animowanymi komunikatami.

Dopuszczalne jest również zrealizowanie własnego pomysłu o porównywalnym zakresie, po wcześniejszym uzgodnieniu go z prowadzącym.

5. Zadania do sprawozdania

W zbiorczym sprawozdaniu z trzech laboratoriów Three.js (Lab 4–6) należy wyraźnie oznaczyć numer laboratorium oraz numery wykonanych zadań.

W części dotyczącej Laboratorium 6, dla każdego z trzech wybranych pomysłów, należy zamieścić:

- Numer i nazwę zrealizowanego rozszerzenia.
- Opis wprowadzonych zmian i zmodyfikowanych plików.
- Najważniejsze fragmenty kodu wraz z krótkim wyjaśnieniem ich działania.
- Zrzut ekranu przedstawiający uzyskany efekt. W przypadku animacji można dołączyć kilka kolejnych klatek.